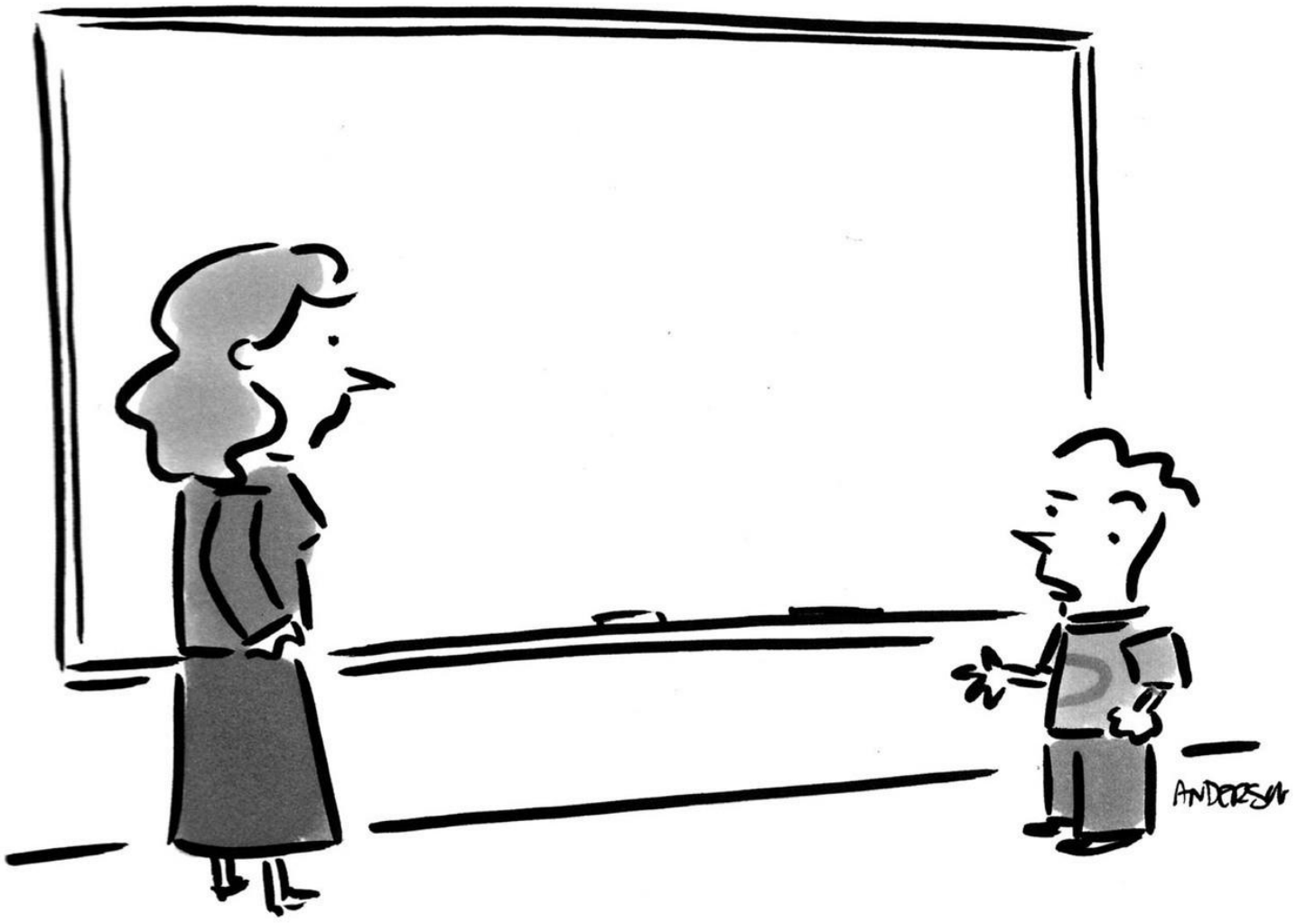
Operating Systems – 3 easy pieces

Chap. 38: Interlude: Files & Directories

Dr. B. Wajid

Monday: Sep. 9, 2024



"Before I write my name on the board, I'll need to know how you're planning to use that data."

Persistent Storage

- Keep a data **intact** even if there is a **power loss**.
  - Hard disk drive (HDD)
  - Solid-state storage device (SDD)
  - RAID storage

- Two key abstractions in the virtualization of storage
  - File
  - Directory

Persistent Storage

- Keep a data **intact** even if there is a **power loss**.
  - Hard disk drive (HDD)
  - Solid-state storage device (SDD)
  - RAID storage

- Two key **abstractions** in the virtualization of storage
  - File
  - Directory

File

- A file is a **stream** (a linear array) of **8-bit bytes**.
- Each file has a **high-level name**, like “myself.txt”
- Each file has a **low-level name (inode number).** The user is not aware of this name.
- Filesystem has a responsibility to store data persistently on disk.

File

- A file is a **stream** (a linear array) of **8-bit bytes**.

- Each file has a **high-level name**, like “myself.txt”

- Each file has a **low-level name (inode number).** The user is not aware of this name.

- Filesystem has a responsibility to store data persistently on disk.

File

- A file is a **stream** (a linear array) of **8-bit bytes**.
- Each file has a **high-level name**, like “myself.txt”
- Each file has a **low-level name (inode number).** The user is not aware of this name.
- Filesystem has a responsibility to store data persistently on disk.

Directory

- Directory is like a file, also has a **high-level** and **low-level name**.
  - It contains a list of **(user-readable name, low-level name)** pairs.
  - The low-level name (**inode number**).
  - Each entry in a directory refers to either *files* or other *directories*.

- Example)
  - A directory has an entry (“**blah_blah_blah**”, “**1000000**”)
    - A file “blah_blah_blah” with the low-level name “1000000.”
    - Notice the file doesn’t have an extension.

  - **/home/bilal/blah_blah_blah**

Directory

- Directory is like a file, also has a **high-level** and **low-level name**.
  - It contains a list of **(user-readable name, low-level name)** pairs.
  - The low-level name (**inode number**).
  - Each entry in a directory refers to either *files* or other *directories*.

- Example)
  - A directory has an entry (“**blah_blah_blah**”, “**1000000**”)
    - A file “blah_blah_blah” with the low-level name “1000000.”
    - Notice the file doesn’t have an extension.

  - **/home/bilal/blah_blah_blah**

Directory

- Directory is like a file, also has a **high-level** and **low-level name**.
  - It contains a list of **(user-readable name, low-level name)** pairs.
  - The low-level name (**inode number**).
  - Each entry in a directory refers to either *files* or other *directories*.

- Example)
  - A directory has an entry ("**blah_blah_blah**", "**1000000**")
    - A file "blah_blah_blah" with the low-level name "1000000."
    - Notice the file doesn't have an extension.

  - **/home/bilal/blah_blah_blah**

Directory

- Directory is like a file, also has a **high-level** and **low-level name**.
  - It contains a list of **(user-readable name, low-level name)** pairs.
  - The low-level name (**inode number**).
  - Each entry in a directory refers to either *files* or other *directories*.

- Examples)
  - A directory has an entry (“**blah_blah_blah**”, “**1000000**”)
    - A file “blah_blah_blah” with the low-level name “1000000.”
    - Notice the file doesn’t have an extension.
  - **/home/bilal/blah_blah_blah**

Directory

- Directory is like a file, also has a **high-level** and **low-level name**.
  - It contains a list of **(user-readable name, low-level name)** pairs.
  - The low-level name (**inode number**).
  - Each entry in a directory refers to either *files* or other *directories*.

- Examples)
  - A directory has an entry (“**blah_blah_blah**”, “**1000000**”)
    - A file **“blah_blah_blah”** with the low-level name **“1000000.”**
    - Notice the file **doesn’t** have an extension.

  - **/home/bilal/blah_blah_blah**

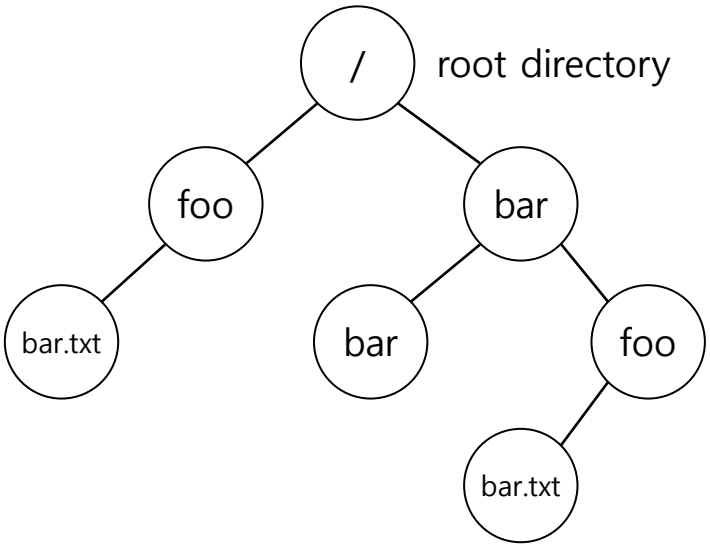
Directory

- Directory is like a file, also has a **high-level** and **low-level name**.
  - It contains a list of **(user-readable name, low-level name)** pairs.
  - The low-level name (**inode number**).
  - Each entry in a directory refers to either *files* or other *directories*.

- Examples)
  - A directory has an entry ("**blah_blah_blah**", "**1000000**")
    - A file "blah_blah_blah" with the low-level name "1000000."
    - Notice the file doesn't have an extension.

  - **/home/bilal/blah_blah_blah**

Directory



An Example Directory Tree

Valid files (absolute pathname) :
/foo/bar.txt
/bar/foo/bar.txt

Valid directory :
/
/foo
/bar
/bar/bar
/bar/foo/

Sub-directories

Creating a file

- Use a system call and appropriate flags to create a file.
  - System call: **open()**,
  - flag: **O_CREAT** flag. (CREAT is without an ‘e’)

- Here we are using three flags:
  - **O_CREAT** : create file.
  - **O_WRONLY** : write only when opened.
  - **O_TRUNC** : truncate file size to zero (remove any existing content).

- **int** fd = **open():** system call returns file descriptor (fd).

- File descriptor is an **int**, and may be considered as a ‘**capability**,’ allowing one to access files.

Creating a file

- Use a system call and appropriate flags to create a file.
  - System call: **open()**,
  - flag: **O_CREAT** flag. (CREAT is without an ‘e’)

- Here we are using three flags:
  - **O_CREAT** : create file.
  - **O_WRONLY** : write only when opened.
  - **O_TRUNC** : truncate file size to zero (remove any existing content).

- **int** fd = **open():** system call returns file descriptor (fd).

- File descriptor is an **int**, and may be considered as a ‘**capability**,’ allowing one to access files.

Creating a file

- Use a system call and appropriate flags to create a file.
  - System call: **open()**,
  - flag: **O_CREAT** flag. (CREAT is without an ‘e’)

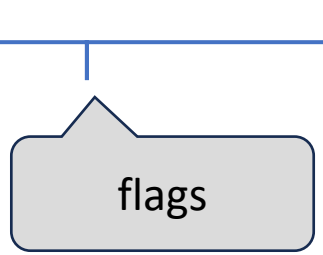
```
int fd = open("foo", O_CREAT | O_WRONLY | O_TRUNC);
```

System call

Creating a file

- Use a system call and appropriate flags to create a file.
  - System call: **open()**,
  - flag: **O_CREAT** flag. (CREAT is without an ‘e’)

```
int fd = open("foo", O_CREAT | O_WRONLY | O_TRUNC);
```



Creating a file

- Use a system call and appropriate flags to create a file.
  - System call: **open()**,
  - flag: **O_CREAT** flag. (CREAT is without an ‘e’)


```
int fd = open("foo", O_CREAT | O_WRONLY | O_TRUNC);
```

Creating a file

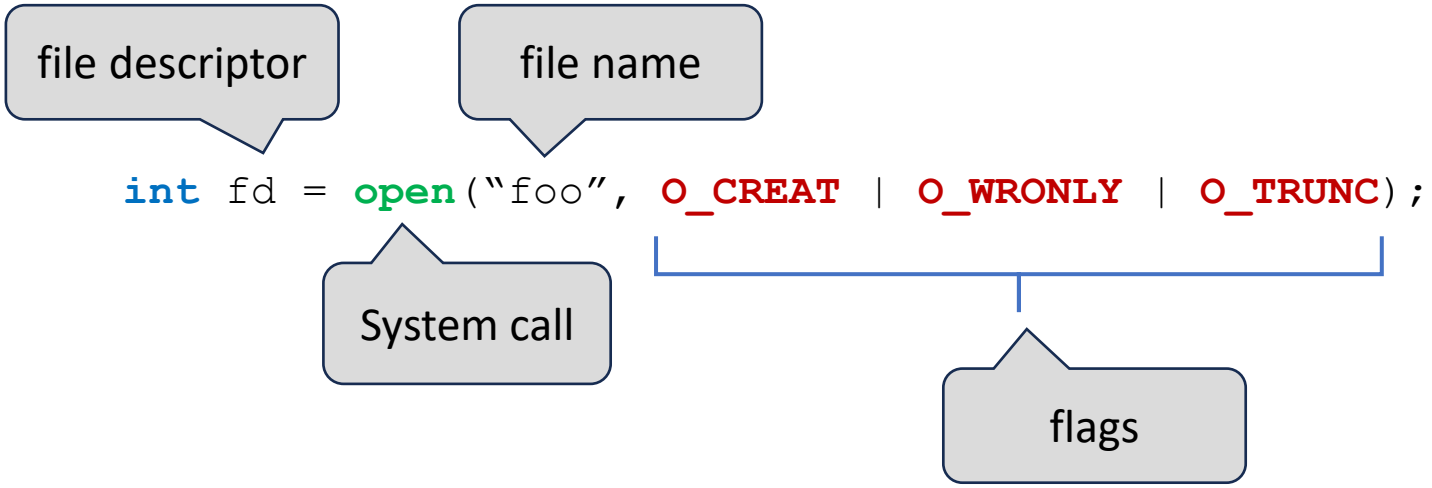
- Use a system call and appropriate flags to create a file.
  - System call: **open()**,
  - flag: **O_CREAT** flag. (CREAT is without an ‘e’)

file descriptor

```
int fd = open("foo", O_CREAT | O_WRONLY | O_TRUNC);
```

Creating a file

- Use a system call and appropriate flags to create a file.
  - System call: **open()**,
  - flag: **O_CREAT** flag. (CREAT is without an ‘e’)



Creating a file

- Use a system call and appropriate flags to create a file.
  - System call: **open()**,
  - flag: **O_CREAT** flag. (CREAT is without an 'e')

```
int fd = open("foo", O_CREAT | O_WRONLY | O_TRUNC);
```

- Here we are using **three flags**:
  - **O_CREAT** : create file.
  - **O_WRONLY** : write only when opened.
  - **O_TRUNC** : truncate file size to zero (remove any existing content).

- **int** fd = **open():** system call returns file descriptor (fd).

- File descriptor is an **int**, and may be considered as a '**capability**,' allowing one to access files.

Creating a file

- Use a system call and appropriate flags to create a file.
  - System call: **open()**,
  - flag: **O_CREAT** flag. (CREAT is without an 'e')

```
int fd = open("foo", O_CREAT | O_WRONLY | O_TRUNC);
```

- Here we are using three flags:
  - **O_CREAT** : create file.
  - **O_WRONLY** : write only when opened.
  - **O_TRUNC** : truncate file size to zero (remove any existing content).

- **int** fd = **open():** system call returns file descriptor (fd).

- File descriptor is an **int**, and may be considered as a '**capability**,' allowing one to access files.

Creating a file

- Use a system call and appropriate flags to create a file.
  - System call: **open()**,
  - flag: **O_CREAT** flag. (CREAT is without an 'e')

```
int fd = open("foo", O_CREAT | O_WRONLY | O_TRUNC);
```

- Here we are using three flags:
  - **O_CREAT** : create file.
  - **O_WRONLY** : write only when opened.
  - **O_TRUNC** : truncate file size to zero (remove any existing content).

- **int** fd = **open():** system call returns file descriptor (fd).

- File descriptor is an **int**, and may be considered as a '**capability**,' allowing one to access files.

Creating a file

- Use a system call and appropriate flags to create a file.
  - System call: **open()**,
  - flag: **O_CREAT** flag. (CREAT is without an 'e')

```
int fd = open("foo", O_CREAT | O_WRONLY | O_TRUNC);
```

- Here we are using three flags:
  - **O_CREAT** : create file.
  - **O_WRONLY** : write only when opened.
  - **O_TRUNC** : truncate file size to zero (remove any existing content).

- **int** fd = **open():** system call returns file descriptor (fd).

- File descriptor is an **int**, and may be considered as a '**capability**,' allowing one to access files.

Creating a file

- Use a system call and appropriate flags to create a file.
  - System call: **open()**,
  - flag: **O_CREAT** flag. (CREAT is without an 'e')

```
int fd = open("foo", O_CREAT | O_WRONLY | O_TRUNC);
```

- Here we are using three flags:
  - **O_CREAT** : create file.
  - **O_WRONLY** : write only when opened.
  - **O_TRUNC** : truncate file size to zero (remove any existing content).

- **int** fd = **open():** system call returns file descriptor (fd).

- File descriptor is an **int**, and may be considered as a '**capability**,' allowing one to access files.

Creating a file

- Use a system call and appropriate flags to create a file.
  - System call: **open()**,
  - flag: **O_CREAT** flag. (CREAT is without an 'e')

```
int fd = open("foo", O_CREAT | O_WRONLY | O_TRUNC);
```

- Here we are using three flags:
  - **O_CREAT** : create file.
  - **O_WRONLY** : write only when opened.
  - **O_TRUNC** : truncate file size to zero (remove any existing content).

- **int** fd = **open():** system call returns file descriptor (fd).
- File descriptor is an **int**, and may be considered as a '**capability**,' allowing one to access files.

Reading & Writing files

- An Example of reading and writing '**foo**' file

```
prompt> echo hello > foo
prompt> cat foo
hello
prompt>
```

- **echo** : redirect the output of echo to the file foo
- **cat** : dump the contents of a file to the screen

Reading & Writing files

- An Example of reading and writing '**foo**' file

```
prompt> echo hello > foo
prompt> cat foo
hello
prompt>
```

  - **echo** : redirect the output of echo to the file foo
  - **cat** : dump the contents of a file to the screen

In this code, we redirect the output of the program to echo to the file foo.

Reading & Writing files

How does the `cat` program access the file foo ?

We can use `strace` to trace the system calls made by a program.

Reading & Writing files

```
prompt> strace cat foo
…
open("foo", O_RDONLY|O_LARGEFILE)  = 3
read(3, "hello\n", 4096)           = 6
write(1, "hello\n", 6)             = 6 // file descriptor 1: standard out
hello
read(3, "", 4096)                  = 0 // 0: no bytes left in the file
close(3)                           = 0
…
prompt>
```

- **Each** running process already has three files open, (i) standard input **[0]**, (ii) standard output **[1]**, and (iii) standard error **[2]**.
- Thus, when you first open another file (as does above), cat it will almost certainly be file descriptor **[3]**.
- "cat" stands for concatenate.

Reading & Writing files

```
prompt> strace cat foo
…
open("foo", O_RDONLY|O_LARGEFILE)  = 3
read(3, "hello\n", 4096)           = 6
write(1, "hello\n", 6)             = 6 // file descriptor 1: standard out
hello
read(3, "", 4096)                  = 0 // 0: no bytes left in the file
close(3)                           = 0
…
prompt>
```

- `open` (file descriptor, flags) = Return file descriptor (3 in example)
- `read` (file des., buffer pointer, size of buffer) = Returns no. of bytes it reads.
- `write` (file desc., buffer pointer, size of buffer) = Returns no. of bytes it writes.
- `close` (3) = closes the file, with file desc. 3.

Reading & Writing files

```
prompt> strace cat foo
…
open("foo", O_RDONLY|O_LARGEFILE)  = 3
read(3, "hello\n", 4096)           = 6
write(1, "hello\n", 6)             = 6 // file descriptor 1: standard out
hello
read(3, "", 4096)                  = 0 // 0: no bytes left in the file
close(3)                           = 0
…
prompt>
```

- `open` (file descriptor, flags) = Return file descriptor (3 in example)
- `read` (file des., buffer pointer, size of buffer) = Returns no. of bytes it reads.
  - file desc. = 3
  - "hello\n" contains 6 characters, 5 for "hello" and 1 for "\n," therefore, return value = 6.
- `write` (file desc., buffer pointer, size of buffer) = Returns no. of bytes it writes.
- `close` (3) = closes the file, with file desc. 3.

Reading & Writing files

```
prompt> strace cat foo
…
open("foo", O_RDONLY|O_LARGEFILE)  = 3
read(3, "hello\n", 4096)           = 6
write(1, "hello\n", 6)             = 6 // file descriptor 1: standard out
hello
read(3, "", 4096)                  = 0 // 0: no bytes left in the file
close(3)                           = 0
…
prompt>
```

- `open` (file descriptor, flags) = Return file descriptor (3 in example)
- `read` (file des., buffer pointer, size of buffer) = Returns no. of bytes it reads.
- `write` (file desc., buffer pointer, size of buffer) = Returns no. of bytes it writes.
  - File desc. = 1, for standard output.
  - "hello\n" is of size 6, 5 for "hello," and 1 for "\n," therefore, return value is 6.
- `close` (3) = closes the file, with file desc. 3.

Reading & Writing files

```
prompt> strace cat foo
…
open("foo", O_RDONLY|O_LARGEFILE)  = 3
read(3, "hello\n", 4096)           = 6
write(1, "hello\n", 6)             = 6 // file descriptor 1: standard out
hello
read(3, "", 4096)                  = 0 // 0: no bytes left in the file
close(3)                           = 0
…
prompt>
```

- `open` (file descriptor, flags) = Return file descriptor (3 in example)
- `read` (file des., buffer pointer, size of buffer) = Returns no. of bytes it reads.
- `write` (file desc., buffer pointer, size of buffer) = Returns no. of bytes it writes.
- `close(3)` = closes the file, with file desc. 3.

Reading And Writing, **But Not Sequentially**

- Up till now, we have been reading/writing files from the beginning to the end.
- What if you want to read from a **random offset** within a document?

- `fd` : File descriptor
- `offset` : Position the file offset to a particular location within the file
- `whence` : Determine how the seek is performed

Reading And Writing, **But Not Sequentially**

- Up till now, we have been reading/writing files from the beginning to the end.

- What if you want to read from a **random offset** within a document?

```
off_t lseek(int fd, off_t offset, int whence);
```

- `fd` : File descriptor
- `offset` : Position the file offset to a particular location within the file
- `whence` : Determine how the seek is performed

Reading And Writing, **But Not Sequentially**

- Up till now, we have been reading/writing files from the beginning to the end.

- What if you want to read from a **random offset** within a document?

```
off_t lseek(int fd, off_t offset, int whence);
```

- **`fd`** : File descriptor.
- **`offset`** : offset in bytes.
- **`whence`** : Determine how the seek is performed

Reading And Writing, **But Not Sequentially**

- Up till now, we have been reading/writing files from the beginning to the end.

- What if you want to read from a **random offset** within a document?

```
off_t lseek(int fd, off_t offset, int whence);
```

- **`fd`** : File descriptor.
- **`offset`** : offset in bytes.
- **`whence`** : Determine how the seek is performed
  - from start (**SEEK_SET**)
  - from current (**SEEK_CUR**)
  - from end (**SEEK_END**)

Reading And Writing, **But Not Sequentially**

System Calls	Return Code	Current Offset
`fd = open("file", O_RDONLY);`	3	0
`read(fd, buffer, 100);`	100	100
`read(fd, buffer, 100);`	100	200
`read(fd, buffer, 100);`	100	300
`read(fd, buffer, 100);`	0	300
`close(fd);`	0	–

How does a **sequential read** occur?

Reading And Writing, **But Not Sequentially**

System Calls	Return Code	OFT[10] Current Offset	OFT[11] Current Offset
`fd1 = open("file", O_RDONLY);`	3	0	–
`fd2 = open("file", O_RDONLY);`	4	0	0
`read(fd1, buffer1, 100);`	100	100	0
`read(fd2, buffer2, 100);`	100	100	100
`close(fd1);`	0	–	100
`close(fd2);`	0	–	–

Let's trace a process that opens the ***same file twice*** and issues a read to each of th em. For example, when a process runs, the file has a unique entry in the open file table. Even if some other process reads the same file at the same time, each will have its own entry in the open file table. In this way, each logical reading or writing of a file is independent, and each has its own current offset while it accesses the given file.

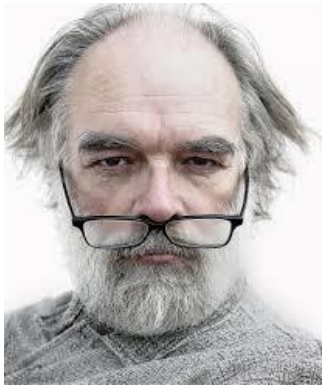
Reading And Writing, **But Not Sequentially**

System Calls	Return Code	Current Offset
`fd = open("file", O_RDONLY);`	3	0
`lseek(fd, 200, SEEK_SET);`	200	200
`read(fd, buffer, 50);`	50	250
`close(fd);`	0	–

Here, the call first sets the current offset to 0. The subsequent lseek() then reads the next 50 bytes, and updates the current offset read() accordingly.

Hey you…

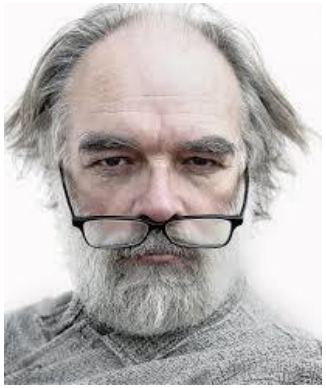
Yes …



Give your students a break

Give your students a break

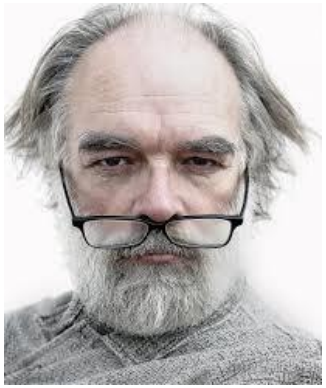
But we just started



GIVE THEM A BREAK !!!!!

GIVE THEM A BREAK !!!!!

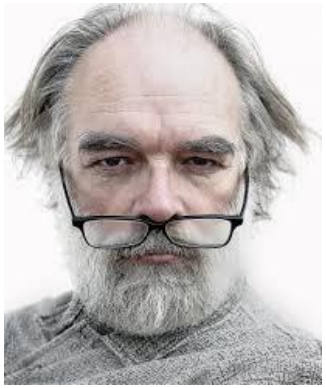
Okayyyyyy.
Its 3 minutes fine



That will do earthling

That will do earthling

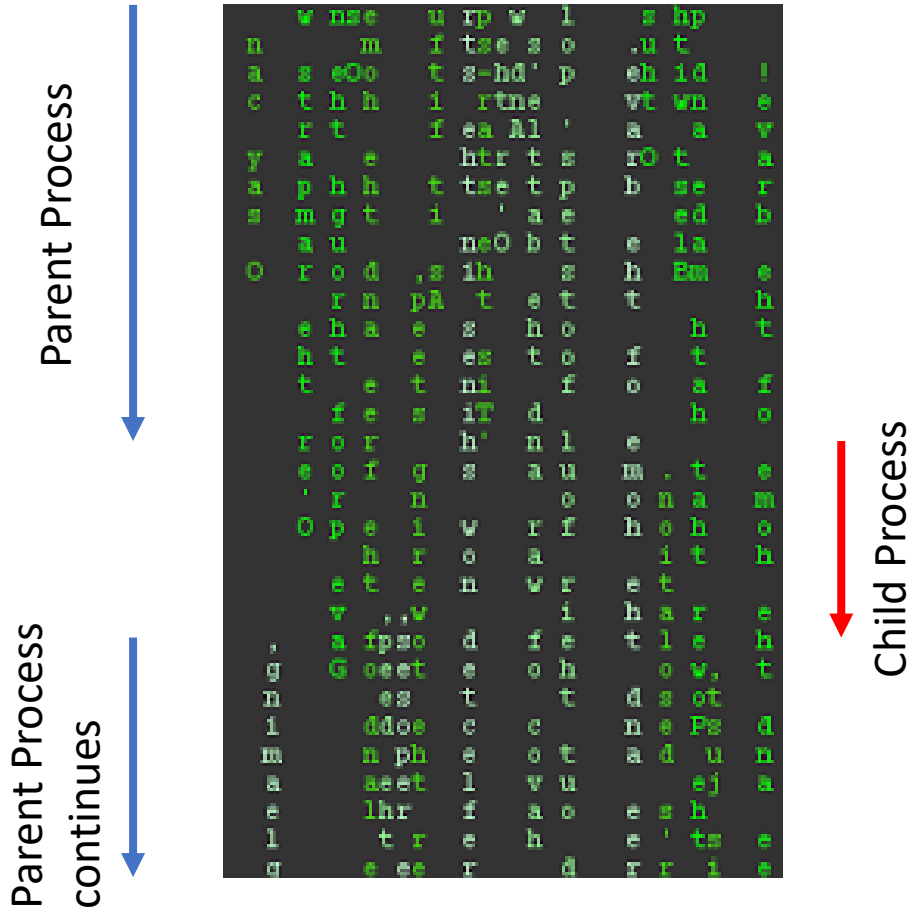
OK.
3 minutes it is



You can thank me later…

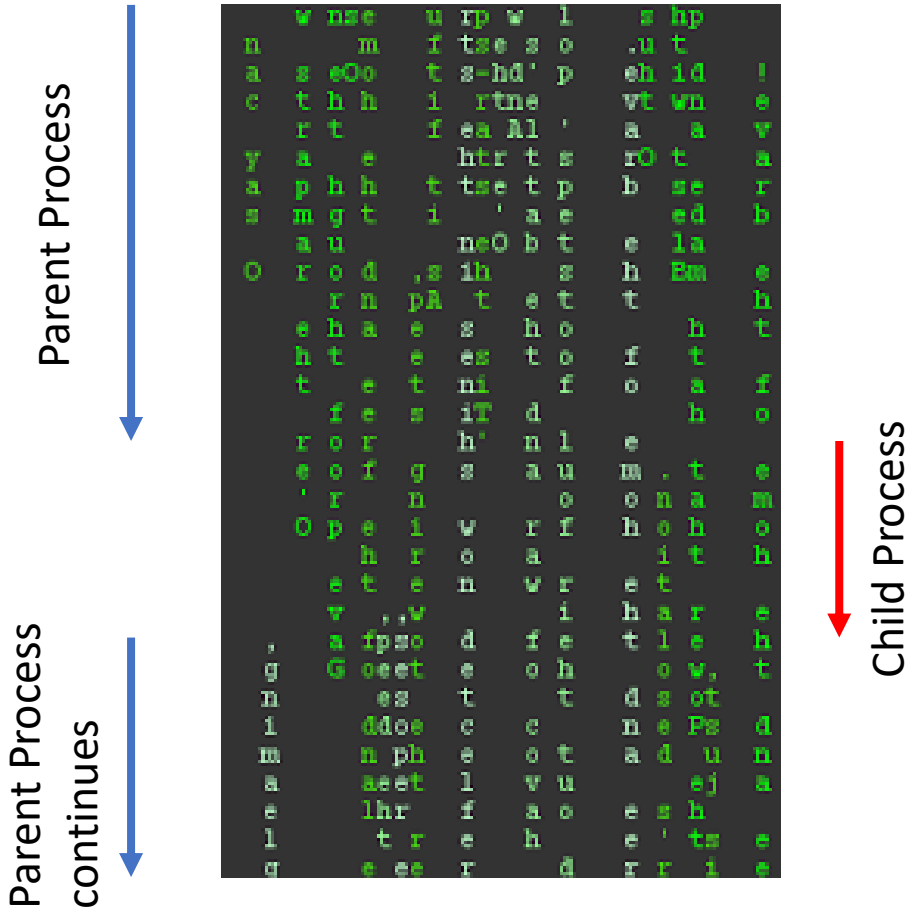
Shared file entries, **fork()**

- Interesting things happen when a ***file table is shared***.
- Here, a parent process creates a child process with fork().
- The parent then waits for the child to complete.
- Finally, the parent, continues from where the child completed its work.



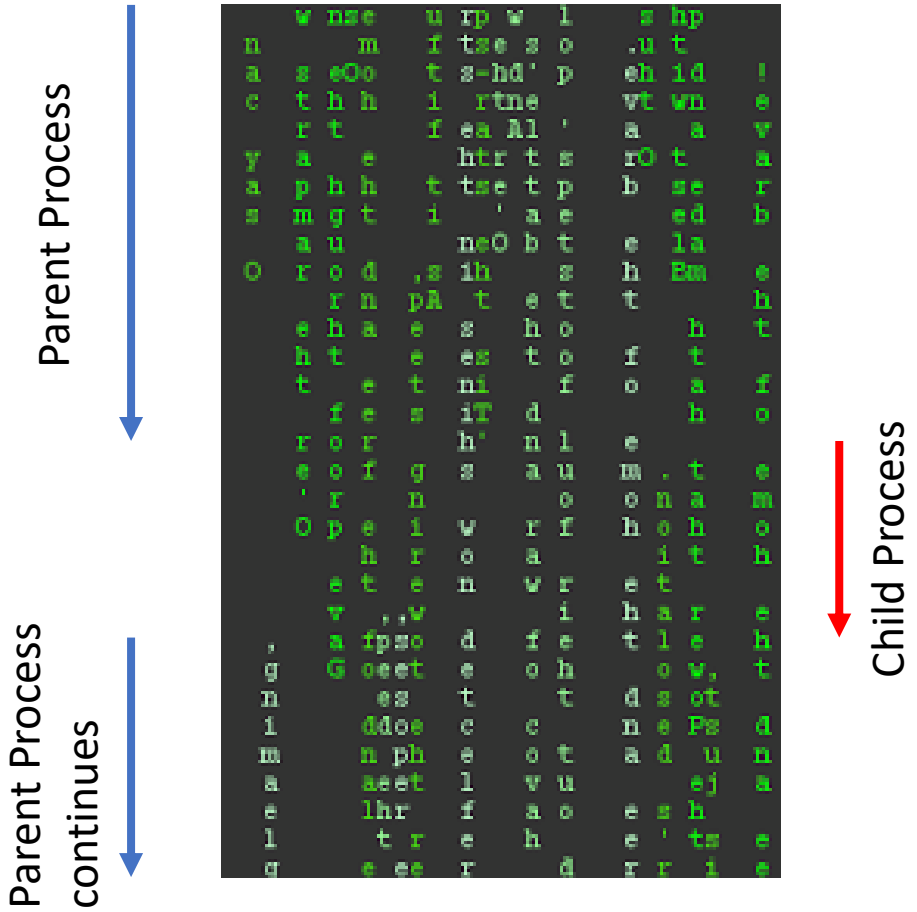
Shared file entries, **fork()**

- Interesting things happen when a ***file table is shared***.
- Here, a parent process creates a child process with fork().
- The parent then waits for the child to complete.
- Finally, the parent, continues from where the child completed its work.



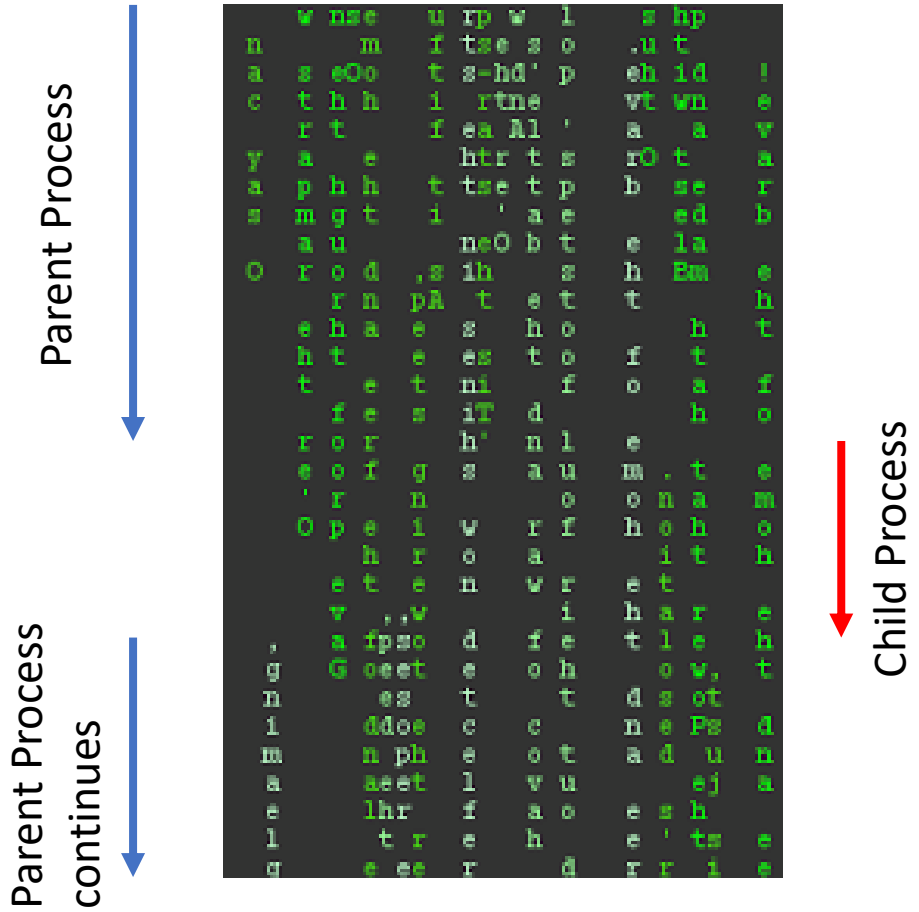
Shared file entries, **fork()**

- Interesting things happen when a ***file table is shared***.
- Here, a parent process creates a child process with fork().
- The parent then waits for the child to complete.
- Finally, the parent, continues from wh ere the child completed its work.



Shared file entries, **fork()**

- Interesting things happen when a ***file table is shared***.
- Here, a parent process creates a child process with fork().
- The parent then waits for the child to complete.
- Finally, the parent, continues from wh ere the child completed its work.



Shared file entries, **fork()**

- Note the
  1. Shared open file table entry,
  2. Shared inode entry,

- If you create several processes that cooperate on a task, they can write to the same output file without additional coordination.

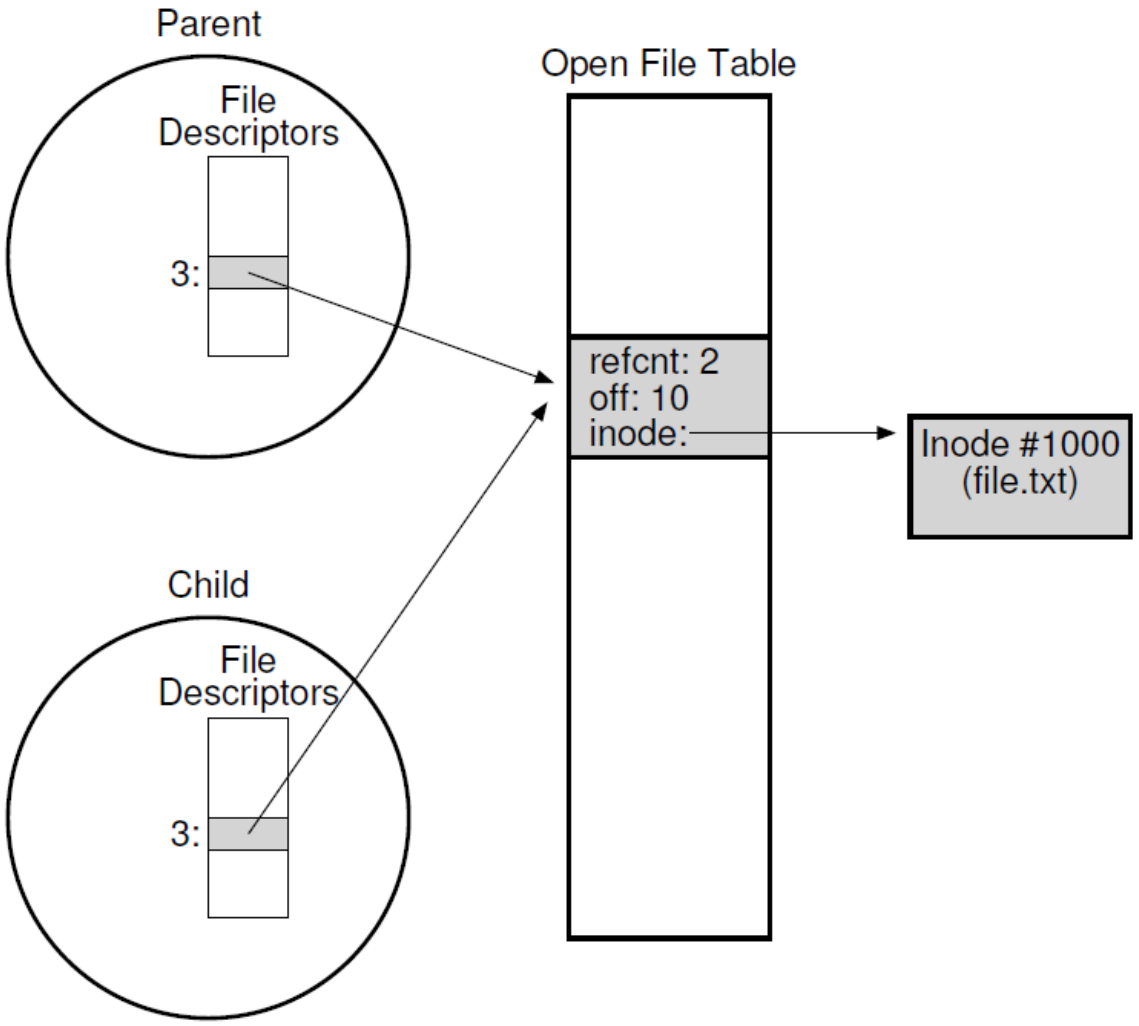


Figure 39.3: **Processes Sharing An Open File Table Entry**

Shared file entries, fork()

```
int main(int argc, char *argv[]) {
    int fd = open("file.txt", O_RDONLY); 1
    assert(fd >= 0);
    int rc = fork();
    if (rc == 0) {
        rc = lseek(fd, 10, SEEK_SET);
        printf("child: offset %d\n", rc);
    } else if (rc > 0) {
        (void) wait(NULL);
        printf("parent: offset %d\n",
                (int) lseek(fd, 0, SEEK_CUR));
    }
    return 0;
}
```

Figure 39.2: **Shared Parent/Child File Table Entries (`fork-seek.c`)**

1. The program attempts to open the file file.txt in read-only mode using the open() system call.

If the file is successfully opened, it returns a file descriptor (fd).

Shared file entries, **fork()**

```
int main(int argc, char *argv[]) {
    int fd = open("file.txt", O_RDONLY);
 2  assert(fd >= 0);
    int rc = fork();
    if (rc == 0) {
        rc = lseek(fd, 10, SEEK_SET);
        printf("child: offset %d\n", rc);
    } else if (rc > 0) {
        (void) wait(NULL);
        printf("parent: offset %d\n",
                (int) lseek(fd, 0, SEEK_CUR));
    }
    return 0;
}
```

Figure 39.2: **Shared Parent/Child File Table Entries (`fork-seek.c`)**

2. If the file doesn't open (for example, if the file doesn't exist), the program would terminate with an assertion failure due to assert(fd >= 0).

Shared file entries, **fork()**

```
int main(int argc, char *argv[]) {
    int fd = open("file.txt", O_RDONLY);
    assert(fd >= 0);
    int rc = fork(); 3
    if (rc == 0) {
        rc = lseek(fd, 10, SEEK_SET);
        printf("child: offset %d\n", rc);
    } else if (rc > 0) {
        (void) wait(NULL);
        printf("parent: offset %d\n",
                (int) lseek(fd, 0, SEEK_CUR));
    }
    return 0;
}
```

Figure 39.2: **Shared Parent/Child File Table Entries (`fork-seek.c`)**

3. The program calls fork() to create a new process.

This creates a child process.fork() returns 0 in the child process and returns the PID (process ID) of the child process in the parent process

Shared file entries, **fork()**

```
int main(int argc, char *argv[]) {
    int fd = open("file.txt", O_RDONLY);
    assert(fd >= 0);
    int rc = fork();
    if (rc == 0) {
        rc = lseek(fd, 10, SEEK_SET);
        printf("child: offset %d\n", rc);
    } else if (rc > 0) {
        (void) wait(NULL);
        printf("parent: offset %d\n",
               (int) lseek(fd, 0, SEEK_CUR));
    }
    return 0;
}
```

4

Figure 39.2: **Shared Parent/Child File Table Entries (`fork-seek.c`)**

4. Child Process Block: In the child process (rc == 0), it moves the file pointer (the current read position in the file), 10 bytes from the beginning of the file using lseek(fd, 10, SEEK_SET).

Then, it prints the current file offset (10) using printf().

Shared file entries, **fork()**

```
int main(int argc, char *argv[]) {
    int fd = open("file.txt", O_RDONLY);
    assert(fd >= 0);
    int rc = fork();
    if (rc == 0) {
        rc = lseek(fd, 10, SEEK_SET);
        printf("child: offset %d\n", rc);
    } else if (rc > 0) {
        (void) wait(NULL);
        printf("parent: offset %d\n",
                (int) lseek(fd, 0, SEEK_CUR));
    }
    return 0;
}
```

5

Figure 39.2: **Shared Parent/Child File Table Entries (`fork-seek.c`)**

5. Parent Process Block: In the parent process (rc > 0), the wait(NULL) function is called, making the parent process wait for the child to finish.

After the child finishes, the parent (i) checks the current position in the file des criptor without changing it, and then (ii) then prints using printf().

Shared file entries, **fork()**

When we run this program, we see the following output:

```
prompt> ./fork-seek
child: offset 10
parent: offset 10
prompt>
```

Renaming Files

- `rename(char* old, char *new)`
  - Rename a file to different name.
  - It implemented as an **atomic call**.
    - Ex) Change from `foo` to `bar`:

```
prompt> mv foo bar          // mv uses the system call rename()
```

    - Ex) How to update a file atomically:

```
int fint fd = open("foo.txt.tmp", O_WRONLY|O_CREAT|O_TRUNC);
write(fd, buffer, size); // write out new version of file
fsync(fd);
close(fd);
rename("foo.txt.tmp", "foo.txt");
```

Getting Information About Files

- `stat()`, `fstat()`: Show the file metadata
  - **Metadata** is information about each file.
  - Ex) Size, Low-level name, Permission, …
  - `stat` structure is below:

```
struct stat {
    dev_t st_dev;         /* ID of device containing file */
    ino_t st_ino;         /* inode number */
    mode_t st_mode;       /* protection */
    nlink_t st_nlink;     /* number of hard links */
    uid_t st_uid;         /* user ID of owner */
    gid_t st_gid;         /* group ID of owner */
    dev_t st_rdev;        /* device ID (if special file) */
    off_t st_size;        /* total size, in bytes */
    blksize_t st_blksize; /* blocksize for filesystem I/O */
    blkcnt_t st_blocks;   /* number of blocks allocated */
    time_t st_atime;      /* time of last access */
    time_t st_mtime;      /* time of last modification */
    time_t st_ctime;      /* time of last status change */
};
```

Getting Information About Files (Cont.)

- To see stat information, you can use the command line tool `stat`.

```
prompt> echo hello > file
prompt> stat file

File: 'file'
Size: 6 Blocks: 8 IO Block: 4096 regular file
Device: 811h/2065d Inode: 67158084 Links: 1
Access: (0640/-rw-r-----) Uid: (30686/ root) Gid: (30686/ remzi)
Access: 2011-05-03 15:50:20.157594748 -0500
Modify: 2011-05-03 15:50:20.157594748 -0500
Change: 2011-05-03 15:50:20.157594748 -0500
```

  - File system keeps this type of information in a `inode` structure.

Removing Files

- `rm` is Linux command to remove a file
  - `rm` call `unlink()` to remove a file.

```
prompt> strace rm foo
…
unlink("foo")                = 0      // return 0 upon success
…
prompt>
```

Why it calls `unlink()`? not "`remove` or `delete`"
We can get the answer later.

Making Directories

- `mkdir()`: Make a directory

```
prompt> strace mkdir foo
…
mkdir("foo", 0777)                    = 0
prompt>
```

  - When a directory is created, it is empty.
  - Empty directory have two entries: . (itself), .. (parent)

```
prompt> ls -a
./      ../
prompt> ls -al
total 8
drwxr-x---  2 remzi remzi    6 Apr 30 16:17 ./
drwxr-x--- 26 remzi remzi 4096 Apr 30 16:17 ../
```

Reading Directories

- A sample code to read directory entries (like `ls`).

```
int main(int argc, char *argv[]) {
    DIR *dp = opendir(".");              // open current directory
    assert(dp != NULL);
    struct dirent *d;
    while ((d = readdir(dp)) != NULL)    // read one directory entry
    {
        // print outthe name and inode number of each file
        printf("%d %s\n", (int) d->d_ino, d->d_name);
    }
    closedir(dp);                        // close current directory
    return 0;
}
```

  - The information available within `struct dirent`

```
struct dirent {
    char            d_name[256];        /* filename */
    ino_t           d_ino;              /* inode number */
    off_t           d_off;              /* offset to the next direct */
    unsigned short  d_reclen;           /* length of this record */
    unsigned char   d_type;             /* type of file */
}
```

Deleting Directories

- `rmdir()`: Delete a directory.
  - Require that the directory be **empty**.
  - If you call `rmdir()` to a non-empty directory, it will fail.
    - I.e., Only has "." and ".." entries.

Hard Links

- `link(old pathname, new one)`
  - **Link** a new file name to an old one
  - Create another way to refer to *the same file*
  - The command-line link program : `ln`

```
prompt> echo hello > file
prompt> cat file
hello
prompt> ln file file2  // create a hard link, link file to file2
prompt> cat file2
hello
```

Hard Links (Cont.)

- The way `link` works:
  - **Create** another name in the directory.
  - **Refer** it to the same inode number of the original file.
    - The file is not copied in any way.
  - Then, we now just have two human names (`file` and `file2`) that both refer to the same file.

Hard Links (Cont.)

- The result of `link()`

```
prompt> ls -i file file2
67158084 file  /* inode value is 67158084 */
67158084 file2 /* inode value is 67158084 */
prompt>
```

  - Two files have **same inode** number, but two human name (file, file2).
  - There is **no difference** between file and file2.
    - Both just links to the underlying metadata about the file.

Hard Links (Cont.)

- Thus, to remove a file, we call `unlink()`.

```
prompt> rm file
removed 'file'
prompt> cat file2          // Still access the file
hello
```

  - ***reference count***
    - Track how many different file names have been linked to this inode.
    - When `unlink()` is called, the reference count decrements.
    - If the reference count reaches zero, the filesystem free the inode and related data blocks. → truly "delete" the file

Hard Links (Cont.)

- The result of `unlink()`
  - `stat()` shows the reference count of a file.

```
prompt> echo hello > file            /* create file*/
prompt> stat file
... Inode: 67158084 Links: 1 ...     /* Link count is 1 */
prompt> ln file file2                /* hard link file2 */
prompt> stat file
... Inode: 67158084 Links: 2 ...     /* Link count is 2 */
prompt> stat file2
... Inode: 67158084 Links: 2 ...     /* Link count is 2 */
prompt> ln file2 file3               /* hard link file3 */
prompt> stat file
... Inode: 67158084 Links: 3 ...     /* Link count is 3 */
prompt> rm file                      /* remove file */
prompt> stat file2
... Inode: 67158084 Links: 2 ...     /* Link count is 2 */
prompt> rm file2                     /* remove file2 */
prompt> stat file3
... Inode: 67158084 Links: 1 ...     /* Link count is 1 */
prompt> rm file3
```

Symbolic Links (Soft Link)

- Symbolic link is more **useful** than Hard link.
  - Hard Link cannot create to a directory.
  - Hard Link cannot create to a file to other partition.
    - Because inode numbers are only unique within a file system.

- Create a symbolic link: `ln -s`

```
prompt> echo hello > file
prompt> ln -s file file2  /* option -s : create a symbolic link, */
prompt> cat file2
hello
```

Symbolic Links (Cont.)

- What is different between *Symbolic link* and *Hard Link*?
  - Symbolic links are **a third type** the file system knows about.

```
prompt> stat file
  ... regular file ...
prompt> stat file2
  ... symbolic link ...        // Actually a file it self of a different type
```

  - The size of symbolic link (`file2`) is **4 bytes**.

```
prompt> ls -al
drwxr-x---  2 remzi remzi   29 May 3 19:10 ./
drwxr-x--- 27 remzi remzi 4096 May 3 15:14 ../              // directory
-rw-r-----  1 remzi remzi    6 May 3 19:10 file             // regular file
lrwxrwxrwx  1 remzi remzi    4 May 3 19:10 file2 -> file    // symbolic link
```

    - A symbolic link holds the pathname of the linked-to file as the data of the link file.

Symbolic Links (Cont.)

- If we link to a longer pathname, our link file would be bigger.

```
prompt> echo hello > alongerfilename
prompt> ln -s alongerfilename file3
prompt> ls -al alongerfilename file3
-rw-r----- 1 remzi remzi  6 May 3 19:17 alongerfilename
lrwxrwxrwx 1 remzi remzi 15 May 3 19:17 file3 -> alongerfilename
```

Symbolic Links (Cont.)

- **Dangling reference**
  - When remove a original file, symbolic link points noting.

```
prompt> echo hello > file
prompt> ln -s file file2
prompt> cat file2
hello
prompt> rm file            // remove the original file
prompt> cat file2
cat: file2: No such file or directory
```

Making and Mounting a File System

- `mkfs` tool : Make a file system
  - Write an empty file system, starting with *a root directory*, on to a disk partition.
  - Input:
    - A device (such as a disk partition, e.g., `/dev/sda1`)
    - A file system type (e.g., `ext3`)

Making and Mounting a File System (Cont.)

- `mount()`
  - Take an existing directory as a target **mount point**.
  - Essentially paste a new file system onto the directory tree at that point.
  - **Example)**

```
prompt> mount -t ext3 /dev/sda1 /home/users
prompt> ls /home/users
a b
```

    - The pathname `/home/users/` now refers to the root of the newly-mounted directory.

Making and Mounting a File System (Cont.)

- `mount` program: show **what is mounted** on a system.

```
/dev/sda1 on / type ext3 (rw)
proc on /proc type proc (rw)
sysfs on /sys type sysfs (rw)
/dev/sda5 on /tmp type ext3 (rw)
/dev/sda7 on /var/vice/cache type ext3 (rw)
tmpfs on /dev/shm type tmpfs (rw)
AFS on /afs type afs (rw)
```

  - `ext3`: A standard disk-based file system
  - `proc`: A file system for accessing information about current processes
  - `tmpfs`: A file system just for temporary files
  - `AFS`: A distributed file system

Acknowledgement: Freepik